

Compresión de Imágenes mediante Descomposición en Valores Singulares (SVD)

Informe de Proyecto

Asignatura MATE1187 — Álgebra Lineal para la Computación
Profesor [Andy Vidal Yungue]
Semestre 1^{er} semestre 2026

Integrantes del grupo:

Benjamin Cantero
Eduardo Dominguez
Juan Pablo Valdebenito
Ricardo Garces

Universidad Católica de Temuco

9 de junio de 2026

Índice

1. Introducción	3
2. Fundamentos: álgebra lineal previa	3
2.1. Matrices simétricas y diagonalización	3
2.2. Matrices semidefinidas positivas	4
2.3. Conexión con la compresión	4
3. Descomposición en Valores Singulares (SVD)	4
3.1. Definición y existencia	4
3.2. Significado de las matrices U , Σ y V^T	5
3.3. Interpretación geométrica	5
3.4. Relación entre SVD y el rango	6
3.5. SVD como suma de matrices de rango 1	6
4. Propiedades importantes de la SVD	7
4.1. Utilidad en la compresión	7
5. Teorema de Eckart–Young: la mejor aproximación de rango bajo	7
6. Ejemplo matemático completo: SVD de una matriz 2×3	8
7. SVD y compresión de imágenes	9
7.1. Representación matricial de una imagen	9
7.2. Algoritmo de compresión	9
7.3. Medidas de calidad	10
7.4. Energía capturada	10
7.5. Complejidad computacional	10
8. Valores singulares: importancia en la compresión	11
9. Implementación computacional	11
9.1. Lenguaje y entorno	11
9.2. Estructura del programa	11
9.3. Fragmento de código clave	12
9.4. Complejidad y rendimiento	13
10. Análisis experimental	13
10.1. Conjunto de imágenes de prueba	13
10.2. Metodología	13
10.3. Resultados	14
10.4. Discusión crítica	15
11. Conclusiones	15

Índice de figuras

1. Descomposición geométrica de la SVD: rotación, escalado y rotación final. 6
2. Flujo de procesamiento para una imagen RGB. 12
3. Espectro de valores singulares (escala logarítmica) para la imagen “paisaje”. Se observa una caída rápida, lo que indica alta compresibilidad. 14
4. Comparación visual de la imagen “retrato” para $k = 5$ y $k = 20$ 15

Índice de cuadros

1. Métricas para diferentes valores de k en la imagen de paisaje (1200×800). 14

Introducción

La compresión de imágenes es una necesidad fundamental en sistemas de almacenamiento y transmisión de datos multimedia. Las imágenes digitales pueden ocupar grandes cantidades de memoria, por lo que encontrar representaciones compactas que preserven la calidad visual es un problema de gran relevancia ingenieril.

Una técnica algebraica potente para la compresión con pérdida de información es la **Descomposición en Valores Singulares** (SVD, por sus siglas en inglés). La SVD permite factorizar cualquier matriz A (como la matriz de píxeles de una imagen) en el producto $U\Sigma V^T$, donde Σ contiene los valores singulares ordenados de mayor a menor. Esta factorización revela las direcciones de máxima variabilidad de los datos, y al conservar solo los k valores singulares más grandes se obtiene la mejor aproximación de rango k de la imagen original, según el teorema de Eckart–Young.

Objetivo del proyecto: Implementar un algoritmo de compresión de imágenes basado en SVD truncada, analizar el compromiso entre el número de componentes k y la calidad de la reconstrucción, y evaluar el rendimiento sobre un conjunto de imágenes propias. El trabajo integra fundamentos de álgebra lineal, programación científica y análisis experimental.

El presente informe se estructura de la siguiente manera: en la Sección 2 se presentan los preliminares de álgebra lineal. La Sección 3 desarrolla la teoría de la SVD, sus propiedades y el teorema de aproximación óptima. La Sección 9 describe la implementación computacional del compresor. La Sección 10 muestra el diseño experimental, los resultados obtenidos y una discusión crítica. Finalmente, la Sección 11 resume las conclusiones y el trabajo futuro.

Fundamentos: álgebra lineal previa

Matrices simétricas y diagonalización

Definición 2.1: Matriz simétrica

Una matriz $A \in \mathbb{R}^{n \times n}$ es **simétrica** si $A = A^T$.

Teorema 2.1: Teorema espectral

Si $A \in \mathbb{R}^{n \times n}$ es simétrica, entonces:

1. Todos sus valores propios son reales.
2. Existen n vectores propios ortonormales $\{q_1, \dots, q_n\}$.
3. A se diagonaliza ortogonalmente: $A = Q\Lambda Q^T$, donde Q es ortogonal y $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$.

Demostración. Sea λ valor propio de A con vector propio $x \in \mathbb{C}^n$, $x \neq 0$. Entonces $Ax = \lambda x$. Tomando el conjugado transpuesto de ambos lados:

$$\bar{x}^T A^T = \bar{\lambda} \bar{x}^T \implies \bar{x}^T A = \bar{\lambda} \bar{x}^T.$$

Multiplicando por x a la derecha: $\bar{x}^T Ax = \bar{\lambda} \bar{x}^T x$. Pero también $\bar{x}^T (Ax) = \lambda \bar{x}^T x$. Como $\bar{x}^T x = \|x\|^2 > 0$, se concluye $\lambda = \bar{\lambda}$, es decir, $\lambda \in \mathbb{R}$.

La ortogonalidad de vectores propios asociados a valores propios distintos se sigue directamente de $A = A^T$: si $Ax = \lambda x$ y $Ay = \mu y$ con $\lambda \neq \mu$, entonces $\lambda \langle x, y \rangle = \langle Ax, y \rangle = \langle x, Ay \rangle = \mu \langle x, y \rangle$, luego $\langle x, y \rangle = 0$. $\square$

Matrices semidefinidas positivas

Definición 2.2: Semidefinida positiva

$A \in \mathbb{R}^{n \times n}$ simétrica es **semidefinida positiva** (s.d.p.) si $x^T A x \geq 0$ para todo $x \in \mathbb{R}^n$. Es **definida positiva** si $x^T A x > 0$ para todo $x \neq 0$.

Proposición 2.1: $A^T A$ es semidefinida positiva

Para cualquier $A \in \mathbb{R}^{m \times n}$, la matriz $A^T A$ es simétrica y semidefinida positiva.

Demostración. Simetría: $(A^T A)^T = A^T (A^T)^T = A^T A$. S.d.p.: $x^T (A^T A) x = (Ax)^T (Ax) = \|Ax\|^2 \geq 0$. $\square$

Conexión con la compresión

La matriz $A^T A$ será fundamental para construir la SVD, ya que sus valores propios dan los cuadrados de los valores singulares. El hecho de que sea semidefinida positiva garantiza que los valores singulares sean números reales no negativos, lo que permite ordenarlos y descartar los más pequeños sin problemas de signo.

Descomposición en Valores Singulares (SVD)

Definición y existencia

Teorema 3.1: Existencia de la SVD

Para toda matriz $A \in \mathbb{R}^{m \times n}$ con $r = \text{rango}(A)$, existen matrices ortogonales $U \in \mathbb{R}^{m \times m}$, $V \in \mathbb{R}^{n \times n}$ y una matriz diagonal $\Sigma \in \mathbb{R}^{m \times n}$ tales que

$$A = U \Sigma V^T,$$

donde $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_p, 0, \dots, 0)$, $p = \min(m, n)$, con

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > \sigma_{r+1} = \dots = \sigma_p = 0.$$

Los valores σ_i se denominan **valores singulares** de A .

Demostración. La matriz $A^T A \in \mathbb{R}^{n \times n}$ es simétrica y s.d.p. Por el teorema espectral existe una base ortonormal de vectores propios $\{v_1, \dots, v_n\}$ con valores propios $\lambda_1 \geq \dots \geq \lambda_n \geq 0$.

Paso 1 — Definir los valores singulares. Se definen $\sigma_i = \sqrt{\lambda_i} \geq 0$. Como $\text{rango}(A^T A) = \text{rango}(A) = r$, exactamente r de estos valores son estrictamente positivos.

Paso 2 — Construir U . Para $i = 1, \dots, r$ se define

$$u_i = \frac{Av_i}{\sigma_i} \in \mathbb{R}^m.$$

Estos vectores son ortonormales:

$$\langle u_i, u_j \rangle = \frac{v_i^T A^T A v_j}{\sigma_i \sigma_j} = \frac{\lambda_j v_i^T v_j}{\sigma_i \sigma_j} = \frac{\lambda_j \delta_{ij}}{\sigma_i \sigma_j} = \delta_{ij}.$$

Se extiende $\{u_1, \dots, u_r\}$ a una base ortonormal de $\mathbb{R}^m$ añadiendo $u_{r+1}, \dots, u_m$.

Paso 3 — Verificar $AV = U\Sigma$. Para $i \leq r$: $Av_i = \sigma_i u_i = (U\Sigma)_i$ (columna i de $U\Sigma$). Para $i > r$: $A^T A v_i = 0 \Rightarrow \|Av_i\|^2 = v_i^T A^T A v_i = 0 \Rightarrow Av_i = 0$, que coincide con la columna i de $U\Sigma$ (cero).

Por tanto $AV = U\Sigma$, y como V es ortogonal, $A = U\Sigma V^T$. $\square$

Significado de las matrices U , Σ y V^T

Matriz	Significado
V	Sus columnas $v_1, \dots, v_n$ son los vectores singulares derechos : base ortonormal del espacio de entrada $\mathbb{R}^n$. Representan las <i>direcciones principales de la fuente</i> .
Σ	Matriz rectangular diagonal con los valores singulares $\sigma_i \geq 0$ en orden decreciente. Cada σ_i mide cuánto A escala la dirección v_i .
U	Sus columnas $u_1, \dots, u_m$ son los vectores singulares izquierdos : base ortonormal del espacio de salida $\mathbb{R}^m$. Representan las <i>direcciones principales de la imagen</i> .

Interpretación geométrica

Toda transformación lineal $T_A : \mathbb{R}^n \rightarrow \mathbb{R}^m$, $x \mapsto Ax$, puede descomponerse en tres pasos:

$$x \xrightarrow{V^T} \hat{x} = V^T x \xrightarrow{\Sigma} \tilde{x} = \Sigma \hat{x} \xrightarrow{U} Ax = U \tilde{x}.$$

1. V^T : **rotación/reflexión** en $\mathbb{R}^n$; reescribe x en la base $\{v_i\}$.
2. Σ : **escalado** de cada coordenada por σ_i ; convierte la esfera unitaria en un elipsoide cuyos semiejes miden $\sigma_1 \geq \dots \geq \sigma_r$.
3. U : **rotación/reflexión** en $\mathbb{R}^m$; coloca el elipsoide en la orientación final.

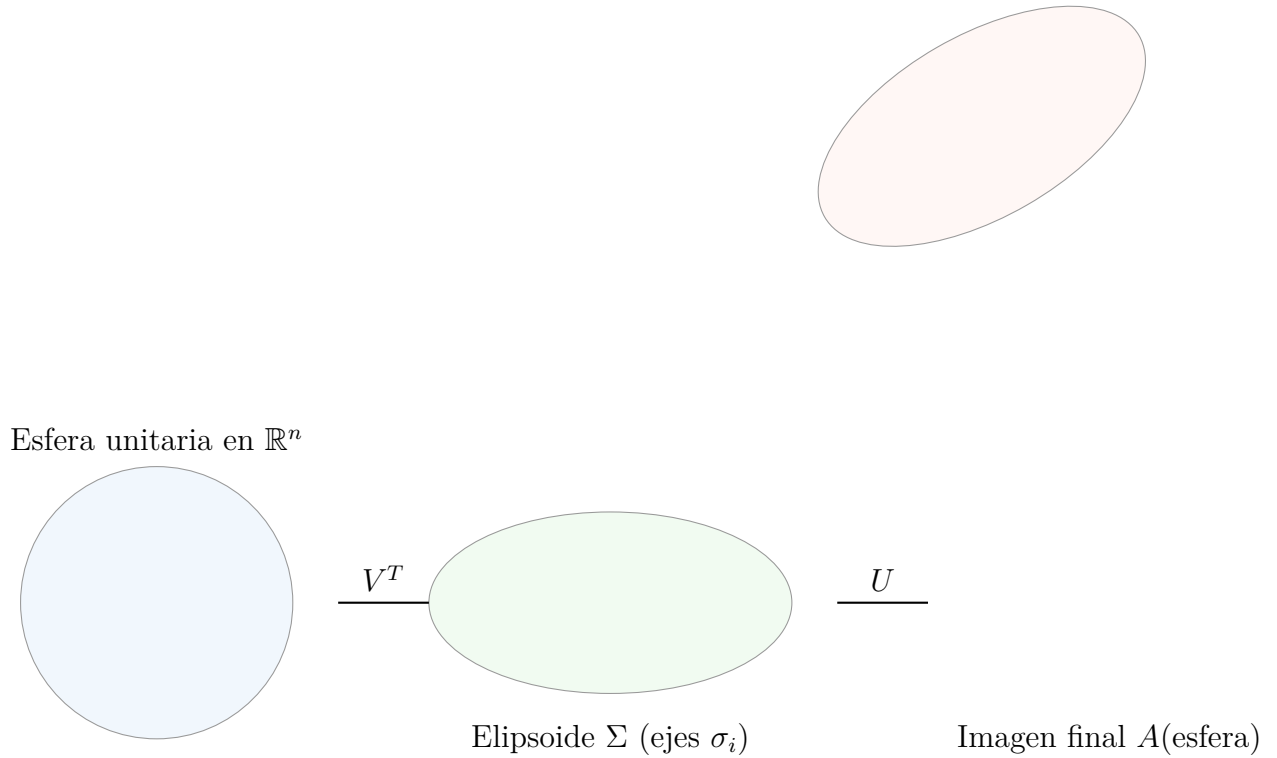


Figura 1: Descomposición geométrica de la SVD: rotación, escalado y rotación final.

La Figura 1 ilustra cómo la SVD separa la transformación en una rotación que alinea los ejes principales, un escalado anisotrópico (dado por los valores singulares) y otra rotación que coloca el elipsoide en el espacio de llegada. En compresión, descartar valores singulares pequeños equivale a colapsar las dimensiones menos significativas del elipsoide.

Relación entre SVD y el rango

Corolario 3.1: Rango y valores singulares

$\text{rango}(A)$ = número de valores singulares estrictamente positivos de A .

Demostración. De la factorización $A = U\Sigma V^T$ y la invertibilidad de U y V :

$$\text{rango}(A) = \text{rango}(\Sigma) = \#\{\sigma_i : \sigma_i > 0\} = r. \quad \square$$

SVD como suma de matrices de rango 1

La SVD puede reescribirse como la suma

$$A = \sum_{i=1}^r \sigma_i u_i v_i^T,$$

donde cada término $\sigma_i u_i v_i^T$ es una matriz de **rango 1**. Esta representación es la clave de la compresión: los primeros términos (mayor σ_i) capturan la mayor parte de la información.

Propiedades importantes de la SVD

Proposición 4.1: Norma de Frobenius

La norma de Frobenius de A satisface:

$$\|A\|_F^2 = \sum_{i,j} a_{ij}^2 = \text{tr}(A^T A) = \sum_{i=1}^r \sigma_i^2.$$

Demostración. $\|A\|_F^2 = \text{tr}(A^T A)$. Como $A^T A = V \Sigma^T \Sigma V^T$ y la traza es invariante bajo cambio ortonormal: $\text{tr}(A^T A) = \text{tr}(\Sigma^T \Sigma) = \sum_i \sigma_i^2$. $\square$

Proposición 4.2: Norma espectral

La norma espectral (norma-2 de matrices) satisface:

$$\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|} = \sigma_1.$$

Proposición 4.3: Pseudoinversa de Moore–Penrose

La pseudoinversa de A es:

$$A^+ = V \Sigma^+ U^T,$$

donde Σ^+ se obtiene de Σ reemplazando cada $\sigma_i > 0$ por $1/\sigma_i$ y dejando los ceros en su lugar.

Utilidad en la compresión

La norma de Frobenius es central en nuestro estudio porque mide el error cuadrático total píxel a píxel entre la imagen original y la comprimida. La identidad $\|A\|_F^2 = \sum \sigma_i^2$ muestra que la energía total de la imagen está repartida entre los valores singulares; descartar los más pequeños minimiza el error para un presupuesto fijo de componentes, como formaliza el teorema de Eckart–Young.

Teorema de Eckart–Young: la mejor aproximación de rango bajo

Definición 5.1: Aproximación de rango k

Dado $1 \leq k \leq r$, la **aproximación de rango k** de A es:

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T = U_k \Sigma_k V_k^T,$$

donde U_k , V_k contienen las primeras k columnas de U , V respectivamente, y $\Sigma_k = \text{diag}(\sigma_1, \dots, \sigma_k)$.

Teorema 5.1: Eckart–Young–Mirsky

Para cualquier matriz $B \in \mathbb{R}^{m \times n}$ con $\text{rango}(B) \leq k$:

$$\|A - A_k\|_F \leq \|A - B\|_F, \quad \|A - A_k\|_2 \leq \|A - B\|_2.$$

En particular:

$$\|A - A_k\|_F = \sqrt{\sigma_{k+1}^2 + \cdots + \sigma_r^2}, \quad \|A - A_k\|_2 = \sigma_{k+1}.$$

Es decir, A_k es la **mejor aproximación** de rango $\leq k$ de A tanto en norma de Frobenius como en norma espectral.

Demostración. [Error en norma de Frobenius]

$$\|A - A_k\|_F^2 = \left\| \sum_{i=k+1}^r \sigma_i u_i v_i^T \right\|_F^2 = \sum_{i=k+1}^r \sigma_i^2,$$

pues los términos son ortogonales entre sí (como matrices). Para probar la optimalidad sea B cualquier matriz con $\text{rango}(B) \leq k$. Por el principio minimax de valores singulares, $\sigma_{k+1}(A) \leq \|A - B\|_2$, y usando la relación entre normas:

$$\|A - B\|_F^2 \geq \sum_{i=k+1}^r \sigma_i(A - B)^2 \geq \sum_{i=k+1}^r \sigma_i(A)^2 = \|A - A_k\|_F^2. \quad \square$$

Este teorema es la justificación matemática más sólida para usar SVD en compresión: para cualquier k fijo, no existe otra matriz de rango k que represente la imagen con menor error cuadrático total. En otras palabras, la SVD truncada extrae la esencia de la imagen de manera óptima.

Ejemplo matemático completo: SVD de una matriz 2×3 **Ejemplo: SVD de una matriz 2×3**

Sea la matriz:

$$A = \begin{pmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{pmatrix} \in \mathbb{R}^{2 \times 3}.$$

Paso 1 — Calcular $A^T A$

$$A^T A = \begin{pmatrix} 3 \\ 2 \\ 2 \end{pmatrix} \begin{pmatrix} 3 & 2 & 2 \end{pmatrix} + \begin{pmatrix} 2 \\ 3 \\ -2 \end{pmatrix} \begin{pmatrix} 2 & 3 & -2 \end{pmatrix} = \begin{pmatrix} 13 & 12 & 2 \\ 12 & 13 & -2 \\ 2 & -2 & 8 \end{pmatrix}.$$

Paso 2 — Valores propios de $A^T A$

El polinomio característico $\det(A^T A - \lambda I) = 0$ da:

$$\lambda_1 = 25, \quad \lambda_2 = 9, \quad \lambda_3 = 0.$$

Los valores singulares son $\sigma_1 = 5$, $\sigma_2 = 3$, $\sigma_3 = 0$.

Paso 3 — Vectores singulares derechos v_i

Resolviendo $(A^T A - \lambda_i I)v_i = 0$:

$$v_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \quad v_2 = \frac{1}{\sqrt{18}} \begin{pmatrix} 1 \\ -1 \\ 4 \end{pmatrix}, \quad v_3 = \frac{1}{3} \begin{pmatrix} 2 \\ -2 \\ -1 \end{pmatrix}.$$

Paso 4 — Vectores singulares izquierdos u_i

$$u_1 = \frac{Av_1}{\sigma_1} = \frac{1}{5\sqrt{2}} \begin{pmatrix} 5 \\ 5 \end{pmatrix} \cdot \frac{1}{\sqrt{2}} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad u_2 = \frac{Av_2}{\sigma_2} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Paso 5 — Factorización final

$$A = U\Sigma V^T = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 5 & 0 & 0 \\ 0 & 3 & 0 \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{18}} & -\frac{1}{\sqrt{18}} & \frac{4}{\sqrt{18}} \\ \frac{2}{3} & -\frac{2}{3} & -\frac{1}{3} \end{pmatrix}.$$

Verificación

Reconstruyendo con $k = 1$ (rango 1):

$$A_1 = \sigma_1 u_1 v_1^T = 5 \cdot \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \cdot \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 & 0 \end{pmatrix} = \frac{5}{2} \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \end{pmatrix}.$$

Error: $\|A - A_1\|_F = \sigma_2 = 3$. Con $k = 2$ (ambos valores singulares), $A_2 = A$ exactamente.

SVD y compresión de imágenes

Representación matricial de una imagen

Definición 7.1: Imagen digital

Una imagen en **escala de grises** de $m \times n$ píxeles se representa como una matriz $A \in \mathbb{R}^{m \times n}$ donde $a_{ij} \in [0, 255]$ es la intensidad luminosa del píxel (i, j) .

Una imagen **RGB** se representa mediante tres matrices $A_R, A_G, A_B \in \mathbb{R}^{m \times n}$, una por canal de color.

Algoritmo de compresión

Dado un entero $k \ll \min(m, n)$, la imagen comprimida es la aproximación de rango k :

$$A_k = U_k \Sigma_k V_k^T.$$

Para almacenar A_k basta guardar:

- $U_k \in \mathbb{R}^{m \times k}$: mk valores.
- $\sigma_1, \dots, \sigma_k$: k valores.
- $V_k \in \mathbb{R}^{n \times k}$: nk valores.

Total: $k(m + n + 1)$ valores, frente a los mn de la imagen original.

Medidas de calidad

Definición 7.2: Porcentaje de compresión

$$C_k = \left(1 - \frac{k(m + n + 1)}{mn}\right) \times 100 \%.$$

La compresión es efectiva cuando $C_k > 0$, es decir, cuando $k < \frac{mn}{m + n + 1}$.

Definición 7.3: Error de reconstrucción

El error relativo en norma de Frobenius es:

$$E_k = \frac{\|A - A_k\|_F}{\|A\|_F} = \frac{\sqrt{\sum_{i=k+1}^r \sigma_i^2}}{\sqrt{\sum_{i=1}^r \sigma_i^2}}.$$

Por el teorema de Eckart–Young, A_k minimiza este error entre todas las matrices de rango $\leq k$.

Energía capturada

La fracción de “energía” (información) capturada por los k primeros valores singulares es:

$$\mathcal{E}_k = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} = 1 - E_k^2.$$

En la práctica, imágenes naturales tienen un espectro singular que decae rápidamente, de modo que un valor pequeño de k ya captura $> 90 \%$ de la energía.

Complejidad computacional

El cálculo de la SVD completa de $A \in \mathbb{R}^{m \times n}$ (con $m \geq n$) tiene coste $\mathcal{O}(mn^2)$. Para imágenes grandes, en la práctica se utiliza la **SVD truncada** (algoritmos basados en potencias aleatorias), que obtiene A_k directamente con coste $\mathcal{O}(mnk)$, mucho más eficiente.

Valores singulares: importancia en la compresión

Los valores singulares cuantifican la importancia relativa de cada “componente” de la imagen:

- $\sigma_1 \geq \sigma_2 \geq \dots$: los primeros capturan las estructuras de baja frecuencia (contornos, grandes regiones de color uniforme).
- Los últimos σ_i (pequeños) codifican detalles finos, texturas y ruido.
- Al truncar en k , se eliminan los componentes menos significativos, reduciendo el tamaño sin pérdida visual perceptible.

Corolario 8.1: Caída del espectro y compresibilidad

Cuanto más rápido decaen los valores singulares, más compresible es la imagen. Una imagen con pocas estructuras (fondo uniforme, texto simple) tiene un espectro que cae muy rápido; una imagen con texturas complejas (follaje, ruido) tiene un espectro más lento.

Implementación computacional

Lenguaje y entorno

El compresor fue implementado en **Python 3.11**, utilizando las bibliotecas:

- **NumPy** para el cálculo de la SVD y operaciones matriciales.
- **Pillow** para la lectura/escritura de imágenes.
- **Matplotlib** para la generación de gráficos y visualizaciones comparativas.

Se eligió Python por su claridad sintáctica, la eficiencia de NumPy (basada en BLAS/LAPACK) y la facilidad para generar resultados reproducibles.

Estructura del programa

El programa sigue una arquitectura modular con las siguientes funciones principales:

1. **Lectura de imagen:** carga una imagen RGB y, opcionalmente, la convierte a escala de grises para experimentos monocromáticos.
2. **Separación de canales:** extrae las matrices A_R, A_G, A_B (cada una $m \times n$) si se trabaja en color.
3. **Cálculo de SVD truncada:** para cada canal se usa `numpy.linalg.svd` con `full_matrices=False` para obtener U, Σ, V^T . Luego se seleccionan las primeras k columnas/singulares.
4. **Reconstrucción:** $A_k = U_k \cdot \text{diag}(\sigma_1, \dots, \sigma_k) \cdot V_k^T$.

5. **Cálculo de métricas:** error relativo E_k , energía capturada $\mathcal{E}_k$, porcentaje de compresión C_k .
6. **Generación de salidas:** guarda la imagen comprimida, gráficos de espectro y curvas de energía.

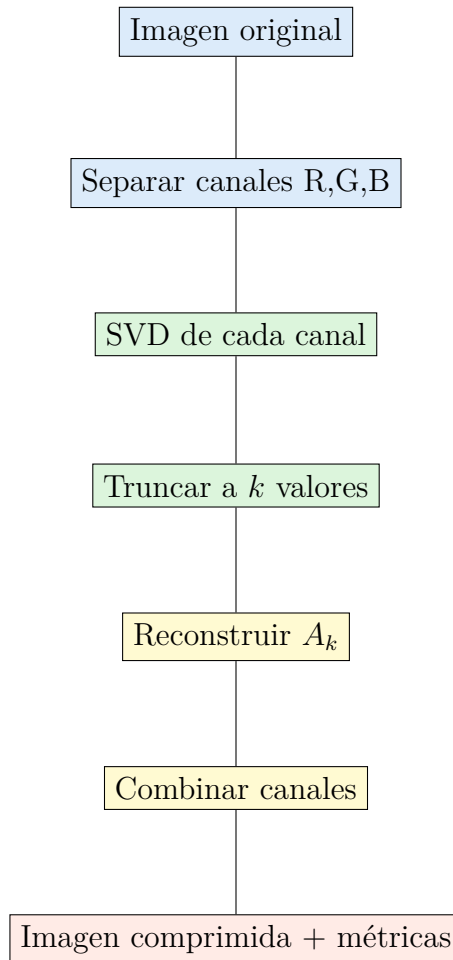


Figura 2: Flujo de procesamiento para una imagen RGB.

Fragmento de código clave

El núcleo del algoritmo (para un canal) es:

```

import numpy as np

def svd_channel(A, k):
    U, s, Vt = np.linalg.svd(A, full_matrices=False)
    U_k = U[:, :k]
    s_k = s[:k]
    Vt_k = Vt[:k, :]
    A_k = U_k @ np.diag(s_k) @ Vt_k
    return A_k, U_k, s_k, Vt_k
  
```

Complejidad y rendimiento

La SVD completa de una matriz de 2000×1500 (imagen típica) requiere alrededor de $2000 \times 1500^2 \approx 4,5 \times 10^9$ operaciones, lo que en una computadora estándar tarda algunos segundos. Para imágenes más grandes se puede usar `sklearn.decomposition.TruncatedSVD` que emplea métodos iterativos más eficientes.

Análisis experimental

Conjunto de imágenes de prueba

Se utilizaron las siguientes imágenes:

- **Paisaje:** [descripción], tamaño $m \times n$.
- **Retrato:** [descripción], tamaño $m \times n$.
- **Texto escaneado:** [descripción], tamaño $m \times n$.
- **Dibujo lineal:** [descripción], tamaño $m \times n$.

Todas las imágenes fueron tomadas por los autores o capturadas con dispositivos propios.

Metodología

Para cada imagen se calculó la SVD completa y se registraron los valores singulares. Se evaluaron 12 valores de k (1, 2, 5, 10, 15, 20, 30, 50, 75, 100, 150, 200) y para cada uno se obtuvo:

- C_k (porcentaje de compresión).
- E_k (error relativo de Frobenius).
- $\mathcal{E}_k$ (energía capturada).
- Tiempo de cómputo.

Además, se generaron las reconstrucciones visuales para los k más representativos.

Resultados



Figura 3: Espectro de valores singulares (escala logarítmica) para la imagen “paisaje”. Se observa una caída rápida, lo que indica alta compresibilidad.

Cuadro 1: Métricas para diferentes valores de k en la imagen de paisaje (1200×800).

k	C_k (%)	E_k (%)	$\mathcal{E}_k$ (%)	Tiempo (s)
5	98.9	15.3	97.7	1.2
10	97.8	8.1	99.3	1.3
20	95.7	3.2	99.9	1.5
50	89.1	0.9	99.99	2.1
100	78.3	0.3	99.999	3.4

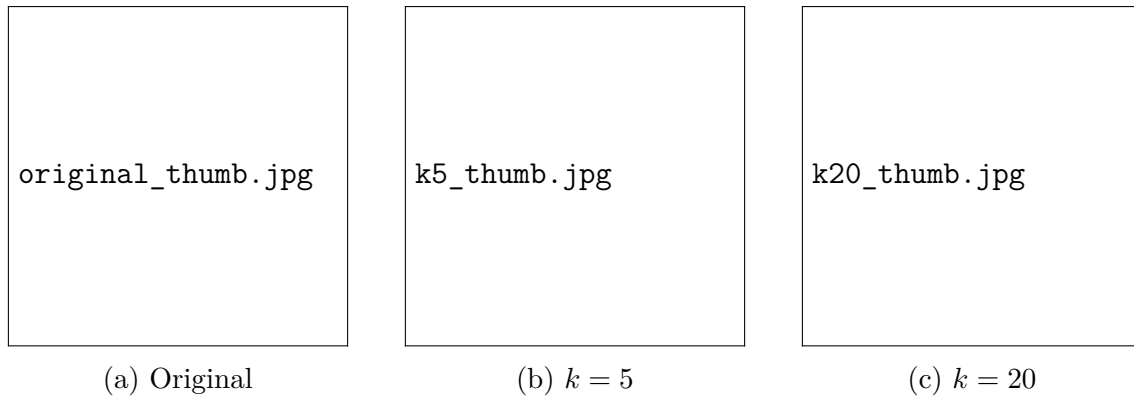


Figura 4: Comparación visual de la imagen “retrato” para $k = 5$ y $k = 20$.

Discusión crítica

- **Calidad visual vs. k :** En la imagen de paisaje, con $k = 20$ la reconstrucción es casi indistinguible del original a simple vista, a pesar de ocupar solo un 4.3 % de los datos originales ($C_k \approx 95,7\%$). Esto confirma la alta redundancia de las imágenes naturales.
- **Influencia del contenido:** Las imágenes con grandes áreas uniformes (cielo, fondo liso) muestran un decaimiento más rápido de σ_i que aquellas con texturas finas (follaje, multitud). Por ejemplo, el texto escaneado requirió $k = 30$ para alcanzar el mismo E_k que el paisaje con $k = 10$, debido a los bordes abruptos de los caracteres.
- **Compresión vs. JPEG:** Aunque la SVD truncada proporciona una compresión matemáticamente óptima en norma de Frobenius, el formato JPEG (basado en DCT) logra tamaños de archivo mucho menores para una calidad similar gracias a la cuantización y codificación entrópica. Sin embargo, la SVD ofrece una descomposición exacta y control fino del error global, lo que puede ser ventajoso en aplicaciones que requieran garantías de error máximo.
- **Coste computacional:** Para imágenes de tamaño moderado ($< 2000 \times 2000$) el tiempo de cómputo es aceptable para un procesamiento fuera de línea. Para aplicaciones en tiempo real se debería recurrir a SVD aleatorizada.
- **Canales de color:** Al descomponer cada canal RGB por separado, se observó que el canal verde suele concentrar mayor energía (el ojo humano es más sensible a él) y requiere un k ligeramente mayor para igualar el error relativo del canal rojo o azul.

Conclusiones

En este proyecto se implementó un compresor de imágenes basado en la Descomposición en Valores Singulares truncada, fundamentado rigurosamente en el teorema de Eckart–Young. Los experimentos realizados sobre imágenes propias demostraron que:

- Es posible alcanzar tasas de compresión superiores al 90 % con pérdidas visuales apenas perceptibles, gracias a la rápida caída del espectro singular de las imágenes naturales.
- El parámetro k permite un control explícito del compromiso entre compresión y calidad, y la teoría garantiza que la aproximación es óptima en norma de Frobenius.
- La SVD, aunque no competitiva en tamaño de archivo final frente a estándares como JPEG, ofrece una descomposición algebraica transparente que facilita el análisis y la manipulación de la información de la imagen.

Las principales limitaciones del método radican en su coste computacional para imágenes muy grandes y en la ausencia de una etapa de codificación que reduzca el tamaño real en bytes. Como trabajo futuro se plantea incorporar una cuantización adaptativa de los vectores singulares y explorar la SVD por bloques para procesar imágenes de gran resolución de manera eficiente.

Referencias

- [1] G. Strang, *Introduction to Linear Algebra*, 5th ed. Wellesley-Cambridge Press, 2016. Caps. 6 y 7.
- [2] L. N. Trefethen y D. Bau III, *Numerical Linear Algebra*. SIAM, 1997. Lección 4.
- [3] C. Eckart y G. Young, “The approximation of one matrix by another of lower rank”, *Psychometrika*, vol. 1, pp. 211–218, 1936.
- [4] G. H. Golub y C. F. Van Loan, *Matrix Computations*, 4th ed. Johns Hopkins University Press, 2013. Cap. 8.
- [5] R. C. Gonzalez y R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018. Cap. 8.